

.NET interview ready pack

For Saurabh | Full-stack developer, approximately 3 years of experience
Interview preparation for 8 October 2026

60 high-value questions with answers, explanations and practical follow-ups. Plus six worked examples, request-flow diagrams, a mock round and a last-minute checklist.

Your goal tonight

Explain the fundamentals clearly, walk through one real feature end to end, and show how you investigate failures. Do not try to memorize every sentence.

How to answer

Definition → **purpose** → **example** → **trade-off**.

Start with a 30-60 second answer. Let the interviewer choose where to go deeper.

Scope

Focused on modern C# and ASP.NET Core Web APIs, with legacy .NET Framework distinctions. These are curated common interview themes, not a measured frequency ranking or a prediction of your interviewer. Code snippets are teaching examples, not a complete deployed application.

Read first if time is short

Questions 7-12, 19-22, 25-33, 37-42, 49-53 and 58-60. Then practise the six worked examples aloud.

Tonight: a 4-hour plan

0:00-0:40 | C# and OOP

Revise value/reference semantics, interfaces, SOLID and collections. Explain one example without looking.

0:40-1:20 | ASP.NET Core

Draw the request pipeline. Rehearse DI lifetimes, authentication, validation and error handling.

1:20-2:00 | Data and async

Review EF Core querying, SQL joins/indexes, transactions and async pitfalls.

2:00-2:10 | Break

Step away from the screen. Then recall five points from memory.

2:10-2:55 | Code and scenarios

Write the Dictionary problem, async method and EF query from the worked examples. Explain assumptions and edge cases.

2:55-3:25 | Your actual project

Describe an API you built: requirement, architecture, database, security, testing, deployment and one trade-off.

3:25-4:00 | Mock interview

Answer the mock round aloud. Revisit only the questions you could not explain. Protect your sleep; avoid an all-night cram.

Tomorrow, 20 minutes: review the final checklist, practise your introduction, and verify the interview link, audio and environment.

C# and runtime fundamentals

Questions 1-3 | Read the answer, cover it, then explain your own example.

01 / What is .NET, and how is it different from C#?

Say this: C# is a programming language. .NET provides the runtime, libraries and tools that run applications written in C# and other supported languages. ASP.NET Core is the web framework.

Example / why: C# source is compiled into an assembly; runtime services include garbage collection and execution. JIT commonly compiles intermediate code to machine code; AOT is another deployment option.

Avoid: Calling C#, .NET and ASP.NET the same thing.

02 / Modern .NET versus .NET Framework?

Say this: Modern .NET is cross-platform and is the usual choice for new ASP.NET Core services. .NET Framework is the older Windows-focused platform used by many existing applications.

Example / why: Ask whether the role uses ASP.NET Core or classic ASP.NET MVC/Web Forms. Migration can involve incompatible dependencies and hosting changes.

Avoid: Assuming a Framework application can always migrate by changing its target version.

03 / What are CLR, managed code and GC?

Say this: The CLR executes managed code and provides services such as memory management, exception handling and type safety. GC reclaims managed objects that are no longer reachable.

Example / why: A static collection that keeps references alive can cause memory growth even with a garbage collector. External resources still require deliberate cleanup.

Avoid: Saying GC prevents every memory leak or immediately frees every unused object.

C# and runtime fundamentals

Questions 4-6 | Read the answer, cover it, then explain your own example.

04 / Value types versus reference types?

Say this: Assigning a value type copies its value. Assigning a reference type copies a reference to the same object. Examples are int/struct and class respectively.

Example / why: Two variables referencing the same List see the same mutations. A struct can contain reference fields, so copying it need not deeply copy the object graph.

Avoid: Saying all value types are on the stack and all references are on the heap.

05 / What is boxing and unboxing?

Say this: Boxing wraps a value type value in an object representation. Unboxing extracts the value with the expected value type. It can create allocations and type-cast failures.

Example / why: object value = 42; int n = (int)value; works. Casting that boxed int directly to long fails; unbox to int first, then convert.

Avoid: Using non-generic collections unnecessarily when a typed collection is available.

06 / Why are strings immutable?

Say this: A string cannot be changed after creation. Operations that appear to modify it produce another string. Immutability makes sharing and reasoning about string values simpler.

Example / why: For repeated concatenation in a loop, StringBuilder can avoid many intermediate strings. For a few values, ordinary interpolation is clearer.

Avoid: Claiming StringBuilder is always faster for every string operation.

OOP and design principles

Questions 7-9 | Read the answer, cover it, then explain your own example.

07 / Explain encapsulation, abstraction, inheritance and polymorphism.

Say this: Encapsulation protects state and valid transitions. Abstraction exposes useful behavior while hiding details. Inheritance derives from a base type. Polymorphism lets different implementations satisfy the same contract.

Example / why: A payment interface abstracts charging; a wallet class guards its balance; card and bank implementations behave differently through the same interface.

Avoid: Reciting definitions without showing how they affect an API or class.

08 / Interface versus abstract class?

Say this: An interface describes a contract and allows multiple interface implementations. An abstract class can share instance state, constructors and implementation; a class has one direct base class.

Example / why: Use `INotificationSender` for email/SMS alternatives. Use an abstract base when a genuine shared base behavior exists. Modern interfaces can also have default members.

Avoid: Saying interfaces can never contain implementations, or inheriting only to reuse a few lines.

09 / Overloading versus overriding?

Say this: Overloading uses the same method name with different parameter signatures. Overriding replaces a virtual or abstract base implementation using runtime polymorphism.

Example / why: `Print(int)` and `Print(string)` are overloads. `Dog` overriding `Animal.Speak` is an override. The `new` keyword hides a member rather than overriding it.

Avoid: Thinking changing only the return type creates an overload.

OOP and design principles

Questions 10-12 | Read the answer, cover it, then explain your own example.

10 / Explain SOLID with practical examples.

Say this: SRP: one reason to change. OCP: extend behavior without repeatedly modifying stable code. LSP: substitutes preserve contracts. ISP: focused interfaces. DIP: high-level logic depends on abstractions.

Example / why: An order service uses IPaymentGateway; a new gateway implementation can be injected. Do not force read-only clients to implement write methods.

Avoid: Creating an interface for every class without a real boundary or testing need.

11 / Composition versus inheritance?

Say this: Composition builds behavior by collaborating with other objects. Inheritance models a substitutable is-a relationship. I prefer composition when capabilities vary independently.

Example / why: An order service has a pricing policy and notification sender. This avoids a growing tree of special-purpose order subclasses.

Avoid: Treating inheritance as the only way to reuse code.

12 / Dependency inversion versus dependency injection?

Say this: Dependency inversion is a design principle about depending on abstractions. Dependency injection is a technique for supplying dependencies from outside an object.

Example / why: A constructor accepts IClock instead of constructing a concrete clock. Production supplies a real clock; a test supplies a predictable one.

Avoid: Confusing the design principle with a particular DI container.

Collections, LINQ and language details

Questions 13-15 | Read the answer, cover it, then explain your own example.

13 / Array, List, Dictionary and HashSet: when?

Say this: Array fits fixed-size indexed data. List supports dynamic indexed sequences. Dictionary maps unique keys to values. HashSet represents unique membership.

Example / why: Use Dictionary<int,int> for Two Sum value-to-index lookup; use HashSet<int> when only duplicate detection matters. Hash lookup is average $O(1)$, not an absolute worst-case guarantee.

Avoid: Using a Dictionary when only membership is needed, or mutating keys in a way that changes their hash.

14 / IEnumerable versus IQueryable?

Say this: IEnumerable represents an enumerable sequence. IQueryable represents a query expression a provider can translate, for example into SQL. Which methods run where depends on the provider and query composition.

Example / why: Filter and project an EF query before ToListAsync. AsEnumerable switches subsequent LINQ operators to in-process enumeration; it does not by itself materialize every row.

Avoid: Assuming IQueryable always means SQL or that every C# expression translates.

15 / What is deferred execution in LINQ?

Say this: Many LINQ operators describe work that runs when the sequence is enumerated. Materializers such as ToList create a result at that moment.

Example / why: Enumerating a database query twice may execute two database calls. Changing the source before enumeration can change the result.

Avoid: Assuming creating a query variable already fetched its rows.

Collections, LINQ and language details

Questions 16-18 | Read the answer, cover it, then explain your own example.

16 / FirstOrDefault, SingleOrDefault and Any?

Say this: FirstOrDefault takes the first match or default. SingleOrDefault requires at most one match and throws if more exist. Any asks only whether a match exists.

Example / why: Use Any for existence checks. Use SingleOrDefault only when uniqueness is a real contract, ideally enforced by a database constraint.

Avoid: Using `Count() > 0` for existence without considering the query cost.

17 / What are delegates, lambdas and events?

Say this: A delegate is a type-safe reference to a compatible method. A lambda is an inline function expression. An event exposes controlled subscription to notifications.

Example / why: A filtering predicate can be a lambda. An event publisher raises a notification to subscribers; long-lived publishers may retain subscribers if they never unsubscribe.

Avoid: Treating every callback as a separate thread.

18 / var, dynamic, nullable types and records?

Say this: var infers a compile-time type. dynamic defers member binding to runtime. Nullable value types represent a value or null. Nullable reference annotations support compiler analysis. Records provide value-oriented equality semantics.

Example / why: `var count = 3` is still an int. A record is useful for a DTO, but a record class remains a reference type and can contain mutable members.

Avoid: Saying var is dynamically typed or that records are automatically deeply immutable.

Async, errors and resource ownership

Questions 19-21 | Read the answer, cover it, then explain your own example.

19 / What do async and await do?

Say this: An async method can suspend at an incomplete await and resume when the awaited operation completes. A Task represents completion, not necessarily a dedicated thread.

Example / why: Awaiting an HTTP call lets the server avoid occupying a thread just waiting for I/O. Marking a CPU loop async does not make its calculations faster.

Avoid: Saying async always starts a new thread. [Reference A]

20 / Why avoid .Result, .Wait() and async void?

Say this: Blocking on asynchronous work can waste threads and may deadlock in synchronization-context environments. Task-returning methods allow callers to await completion and observe exceptions; async void is mainly for event handlers.

Example / why: An ASP.NET Core request should await its data call rather than block on Result. Thread-pool starvation can still occur even without the classic UI deadlock.

Avoid: Fire-and-forget work that silently fails after a request finishes. [A]

21 / Task.WhenAll and CancellationToken?

Say this: WhenAll combines completion of independent tasks. CancellationToken communicates a cooperative cancellation request that operations must observe or pass onward.

Example / why: Parallelize independent remote calls within limits. Pass cancellation to HTTP and EF methods. Never start parallel EF operations on one DbContext.

Avoid: Assuming cancellation forcibly terminates all code or that unbounded concurrency is safe. [A, E]

Async, errors and resource ownership

Questions 22-24 | Read the answer, cover it, then explain your own example.

22 / Task.Run versus naturally asynchronous I/O?

Say this: Task.Run schedules work on the thread pool, which can help keep a desktop UI responsive during CPU work. Naturally asynchronous I/O does not need Task.Run around it.

Example / why: For expensive server jobs, use a bounded background worker or queue with appropriate capacity rather than spawning unlimited request-side tasks.

Avoid: Wrapping every database or HTTP call in Task.Run.

23 / How should you handle exceptions?

Say this: Catch exceptions where you can recover or add useful context. Use centralized API error handling for unexpected failures. Log diagnostic details internally and return a safe, consistent client error.

Example / why: Use throw; to rethrow while preserving stack information. Handle expected validation failures explicitly instead of using exceptions as normal control flow.

Avoid: Returning stack traces, connection strings or secrets to clients.

24 / IDisposable, using and garbage collection?

Say this: IDisposable provides deterministic resource cleanup. using ensures Dispose is called when leaving scope, including on exceptions. GC handles managed memory and is not a substitute for timely resource release.

Example / why: Use using for streams you own; await using for IAsyncDisposable resources. Let the DI container dispose services it creates within their lifetime.

Avoid: Calling Dispose on injected shared dependencies you do not own.

ASP.NET Core fundamentals

Questions 25-27 | Read the answer, cover it, then explain your own example.

25 / Walk through an ASP.NET Core request.

Say this: The server receives the request, then middleware processes it in order. Routing selects an endpoint; authentication and authorization run as configured; the endpoint handles input and returns a response.

Example / why: Response processing unwinds through middleware that awaited the next delegate. A middleware can short-circuit the pipeline. See the flow page.

Avoid: Assuming every request always reaches a controller. [B]

26 / Why does middleware order matter?

Say this: Earlier middleware wraps later middleware. Exception handling must be placed where it can catch downstream failures. Authentication should precede authorization.

Example / why: A typical protected API explicitly orders routing, CORS when needed, authentication and authorization before executing its mapped endpoints. Static/public endpoints need deliberate policy decisions.

Avoid: Copying one pipeline order without considering endpoint and hosting requirements. [B]

27 / Transient, scoped and singleton lifetimes?

Say this: Transient creates a new instance per resolution. Scoped reuses an instance within a scope, commonly an HTTP request. Singleton reuses one instance for the container lifetime.

Example / why: Choose a scoped business service when it uses the request DbContext. A singleton must handle concurrent access safely.

Avoid: Thinking scoped means one instance for the entire application. [C]

ASP.NET Core fundamentals

Questions 28-30 | Read the answer, cover it, then explain your own example.

28 / Why cannot a singleton capture a scoped DbContext?

Say this: Its lifetime outlives the request scope, so it can retain the wrong context and share unsafe state across requests. Lifetime validation may reject this configuration.

Example / why: A background worker creates and disposes an explicit scope for each unit of work. It does not keep the scoped object after disposing that scope.

Avoid: Changing DbContext to singleton to silence a lifetime error. [C]

29 / Controllers versus Minimal APIs?

Say this: Both define HTTP endpoints. Controllers offer an attribute-based organization and MVC features. Minimal APIs provide concise endpoint mapping and can also use services, filters and route groups.

Example / why: Choose based on application size, conventions and team needs. Business logic belongs in testable services rather than large endpoint bodies.

Avoid: Calling one approach inherently secure or production-ready and the other not.

30 / Model binding, validation and DTOs?

Say this: Binding converts request values into parameters or models. Validation checks whether input meets rules. DTOs define the external contract separately from persistence entities.

Example / why: An update DTO exposes allowed fields rather than accepting an entire User entity with an IsAdmin property. Controller APIs using ApiController can produce automatic validation responses.

Avoid: Assuming all endpoint styles and framework versions have identical automatic validation behavior.

API security and HTTP

Questions 31-33 | Read the answer, cover it, then explain your own example.

31 / Authentication versus authorization?

Say this: Authentication determines who the caller is. Authorization determines what that caller may do. An authenticated user is not automatically allowed to access every resource.

Example / why: A valid token identifies a user; an ownership check still ensures the requested order belongs to that user. Policies can express permission requirements.

Avoid: Checking only whether the user is logged in.

32 / How does JWT validation work?

Say this: A bearer token is sent with the request. The API validates signature, issuer, audience and lifetime according to configured trust. Claims are used for authorization.

Example / why: Signed JWT payloads are usually readable, not encrypted. Use HTTPS and avoid storing secrets in claims. Revocation/refresh design needs separate consideration.

Avoid: Merely decoding a JWT and trusting its payload.

33 / Which HTTP status codes should an API use?

Say this: 200 for successful content, 201 for creation, 204 for successful no-content, 400 for invalid requests, 401 for missing/invalid authentication, 403 for denied access, 404 for not found, 409 for conflicts and 500 for unexpected failures.

Example / why: A created resource can return a Location header. A validation error should identify safe, actionable field errors.

Avoid: Returning 200 for every failure or exposing internal exception messages.

API security and HTTP

Questions 34-36 | Read the answer, cover it, then explain your own example.

34 / CORS versus CSRF?

Say this: CORS controls browser cross-origin access to responses; it is not authentication. CSRF exploits automatically attached credentials, such as cookies, to trigger unwanted actions.

Example / why: Cookie-authenticated applications need an appropriate anti-CSRF strategy. Allowed origins should be deliberate; bearer token designs have different exposure, including XSS risks.

Avoid: Treating CORS as a firewall that prevents all non-browser callers.

35 / How do you prevent SQL injection and overposting?

Say this: Use parameterized database queries and validate allowed input. Use explicit DTOs and server-side authorization for changes. Never trust client-supplied role or ownership fields.

Example / why: Interpolated SQL APIs may parameterize values, but raw string concatenation is dangerous. Dynamic identifiers require allowlisting because parameters represent values, not SQL structure.

Avoid: Assuming an ORM makes every raw SQL string safe.

36 / PUT, PATCH, POST and idempotency?

Say this: PUT generally replaces a resource representation; PATCH applies partial changes; POST commonly creates or invokes an action. Idempotency means repeating a request has the same intended effect as doing it once.

Example / why: A payment API can persist an idempotency key with a request fingerprint and outcome under a uniqueness constraint. PATCH idempotency depends on its operation.

Avoid: Retrying a charge blindly or confusing idempotency with identical response codes.

EF Core and database access

Questions 37-39 | Read the answer, cover it, then explain your own example.

37 / What does DbContext do?

Say this: DbContext coordinates queries, tracked entity changes and persistence in EF Core. It represents a short-lived unit of work and is not safe for concurrent use.

Example / why: Load an order, change a property, then SaveChangesAsync. Keep request contexts scoped; use a separate scope/context for a separate background operation.

Avoid: Sharing one context across threads. [E]

38 / Tracking versus AsNoTracking?

Say this: Tracking queries retain entity change information for later updates. AsNoTracking avoids that tracking overhead for read-only entity results.

Example / why: For a read-only listing, project only needed columns into DTOs. Tracking is useful for a read-modify-save operation, so disable it deliberately rather than everywhere.

Avoid: Believing AsNoTracking makes an inefficient query automatically fast. [D]

39 / What is the N+1 query problem?

Say this: One query fetches a list, then an additional query runs for each item to load related data. Latency grows with the list size.

Example / why: Inspect SQL and use appropriate projection or eager loading. Large multi-collection joins can also over-fetch, so assess split queries and pagination when appropriate.

Avoid: Adding Include for every navigation without measuring result size. [D]

EF Core and database access

Questions 40-42 | Read the answer, cover it, then explain your own example.

40 / How do you handle concurrent updates?

Say this: Optimistic concurrency compares an original concurrency token during an update. If another writer changed the row, the update can fail with `DbUpdateConcurrencyException`.

Example / why: Use a SQL Server rowversion or another configured token. Reload/merge, reject with a conflict, or retry only when business semantics allow it.

Avoid: Silently overwriting another user or assuming every database has SQL Server rowversion. [F]

41 / What are migrations?

Say this: Migrations describe incremental schema changes derived from model changes. Review the generated operations and apply them through a controlled deployment process.

Example / why: Adding a required column to a populated table may need a staged backfill. Plan backup, compatibility and rollback/forward-fix before production changes.

Avoid: Automatically applying destructive migrations from every application instance.

42 / How do you improve a slow database endpoint?

Say this: Measure timings and inspect generated SQL and the database execution plan. Reduce rows and columns, use appropriate indexes, avoid N+1 calls and page large results.

Example / why: Filter before materialization; use a stable unique ordering for pagination. Caching may help repeated reads but introduces invalidation decisions.

Avoid: Guessing that more application servers will fix a poorly indexed query. [D]

SQL and transactions

Questions 43-45 | Read the answer, cover it, then explain your own example.

43 / INNER JOIN versus LEFT JOIN?

Say this: INNER JOIN returns matching rows. LEFT JOIN also retains unmatched left-side rows with null values on the right. One-to-many matches can produce multiple result rows.

Example / why: List customers with no orders using a left join and an IS NULL check on the non-nullable order key. Moving right-side filters into WHERE can remove unmatched rows.

Avoid: Assuming a join always returns one result per left row.

44 / WHERE, HAVING and GROUP BY?

Say this: WHERE filters rows before grouping. GROUP BY forms groups. HAVING filters groups based on aggregate results or grouping expressions.

Example / why: Find customers with at least three paid orders: filter status in WHERE, group by customer, then HAVING COUNT(*) >= 3.

Avoid: Using HAVING as a replacement for every ordinary row filter.

45 / What is an index, and what does it cost?

Say this: An index is an additional access structure that can speed suitable lookups, joins and ordering. It uses storage and adds maintenance cost to writes.

Example / why: A composite index can help a query filtering by customer and ordering by creation time; column order and database behavior matter. Check the execution plan.

Avoid: Indexing every column or promising every indexed query is fast.

SQL and transactions

Questions 46-48 | Read the answer, cover it, then explain your own example.

46 / Explain ACID.

Say this: Atomicity: commit all work or none. Consistency: preserve defined invariants. Isolation: control interference between transactions. Durability: committed data survives according to the database guarantees.

Example / why: A bank transfer updates both balances in one transaction and enforces sufficient funds. Isolation levels affect which concurrent anomalies are possible.

Avoid: Confusing transaction consistency with distributed CAP consistency.

47 / Normalization versus denormalization?

Say this: Normalization organizes facts to reduce duplication and update anomalies. Denormalization deliberately duplicates or precomputes data for a measured read requirement.

Example / why: Separate customers from orders; add a reporting summary only if its performance benefit justifies keeping that summary consistent.

Avoid: Assuming joins are inherently bad or duplicate data is always wrong.

48 / Deadlock versus optimistic concurrency conflict?

Say this: A deadlock is a cycle of waiting for resources; the database usually chooses a victim. An optimistic conflict means a version check detected another update.

Example / why: Reduce deadlocks with short transactions and consistent access order. A bounded retry may be appropriate, but a concurrency conflict can require user resolution.

Avoid: Retrying every database exception forever.

Production and system scenarios

Questions 49-51 | Read the answer, cover it, then explain your own example.

49 / An endpoint is slow in production. What do you do?

Say this: Define the symptom and compare latency percentiles, traffic and errors. Trace a request across API, database and dependencies. Identify whether time is CPU, I/O, locking or queueing.

Example / why: Check recent deployments and query plans before changing code. Make one measured fix, then verify latency and correctness under representative load.

Avoid: Starting with random caching or rewriting the entire service.

50 / How should you use HttpClient?

Say this: Reuse connection management through a suitable long-lived client/handler or IHttpClientFactory. Configure timeouts and pass cancellation. Treat remote failures explicitly.

Example / why: A typed client can centralize an external API integration. Retry transient failures with limits; unsafe operations need idempotency before retries.

Avoid: Creating and disposing a new handler for every request or retrying payments blindly.

51 / When would you use Redis?

Say this: Redis can provide shared low-latency caching or other data structures across application instances. A cache-aside flow reads cache, falls back to the database, then populates cache.

Example / why: Set a TTL and define invalidation after writes. Plan for cache outage and hot-key/stampede behavior. Keep the authoritative data source explicit.

Avoid: Treating cached data as permanently correct or adding a cache before measuring.

Production and system scenarios

Questions 52-54 | Read the answer, cover it, then explain your own example.

52 / What is a background queue useful for?

Say this: It moves slow or bursty work out of the interactive request and allows bounded processing and retries. Return a job identifier when work is asynchronous.

Example / why: Generate a report in a worker, persist job status and let the client poll. Make consumers idempotent because messages may be delivered more than once.

Avoid: Claiming a queue alone guarantees exactly-once business effects.

53 / Explain a layered API architecture.

Say this: An endpoint handles HTTP concerns, an application service coordinates a use case, domain logic enforces rules, and infrastructure handles storage or external systems.

Example / why: Place business validation where it can be reused and tested. Small services can stay simple; extra layers should justify their maintenance cost.

Avoid: Adding repositories and interfaces mechanically when they hide useful EF capabilities without benefit.

54 / Unit tests versus integration tests?

Say this: Unit tests isolate small behavior with controlled dependencies. Integration tests verify collaborating parts such as routing, authentication, serialization and database behavior.

Example / why: Test a pricing rule in isolation; test an API update against a representative database provider. EF in-memory behavior is not a substitute for relational semantics.

Avoid: Mocking everything and never testing real persistence or authorization.

Deployment, teamwork and interview communication

Questions 55-57 | Read the answer, cover it, then explain your own example.

55 / How would you deploy a .NET API with Docker and CI/CD?

Say this: Build and test in CI, create a versioned image, scan/configure it and deploy with health checks and rollback capability. A multi-stage image separates build tools from runtime.

Example / why: Provide secrets at runtime and handle schema changes separately and safely. Confirm readiness before shifting traffic. Docker packages an app; it is not a virtual machine per container.

Avoid: Baking secrets into image layers or assuming a passing build proves production health.

56 / Logs, metrics and traces: why all three?

Say this: Logs describe events, metrics show aggregate trends, and traces connect work across a request path. Together they help identify impact and root cause.

Example / why: Use correlation identifiers and structured fields for an order request. Track latency, error rates and saturation; redact personal data and credentials.

Avoid: Logging entire request bodies or access tokens by default.

57 / HLD versus LLD for an order service?

Say this: HLD covers components, data flow, scaling and reliability. LLD covers interfaces, classes, schemas and detailed interactions within those components.

Example / why: Start with requirements; draw client, API, database and queue. Then detail PlaceOrder, stock checks, transaction boundaries and idempotency.

Avoid: Drawing microservices before asking about traffic, consistency and team constraints.

Deployment, teamwork and interview communication

Questions 58-60 | Read the answer, cover it, then explain your own example.

58 / Tell me about yourself.

Say this: I am a full-stack developer with around three years of experience. I work on [your real stack] and have contributed to [real feature]. My strengths are [evidence]. I am looking to deepen backend ownership in this role.

Example / why: Keep it to 60-90 seconds. Connect one feature, one technical decision and one outcome to the job. If .NET is new, say so and explain transferable backend experience.

Avoid: Inventing production .NET experience, company names or impact numbers.

59 / Describe a production issue or disagreement.

Say this: Use STAR: situation, task, action and result. Explain what you personally investigated or changed, how you collaborated and what you learned.

Example / why: For a disagreement: state the shared goal, evidence, trade-off and small experiment. Use a real example; mention the prevention step after an incident.

Avoid: Blaming teammates or claiming team work as entirely your own.

60 / What if you do not know an answer?

Say this: State what you know, clarify assumptions, and reason through the problem. Be honest about the gap and describe how you would verify the detail.

Example / why: Say: I have not used that feature in production. My understanding is ... I would verify it with a small example and the official documentation.

Avoid: Guessing confidently. Ask useful closing questions about their .NET version, database, testing and ownership expectations.

Picture the request

A simplified protected API flow. Actual middleware depends on the application.

01 Browser / mobile client

Sends HTTPS request with JSON and appropriate credentials.

| next |

02 Middleware pipeline

Exception handling wraps downstream work. Routing selects an endpoint; authentication identifies the caller; authorization checks permission.

| next |

03 Endpoint + DTO validation

Validate shape and rules. Check ownership. Translate HTTP input into a use case.

| next |

04 Service + database

Run business logic; query/update via a scoped DbContext. Keep the transaction boundary intentional.

| next |

05 Response

Map to a response DTO and correct status. Response travels back through the pipeline.

| next |

DI memory picture: singleton = shared application object; scoped = one object per scope; transient = a fresh object each resolution. These are ownership/lifetime rules, not thread-safety guarantees.

Worked example 01 / Dictionary: Two Sum

```
static int[] TwoSum(int[] nums, int target)
{
    var seen = new Dictionary<int, int>();
    for (int i = 0; i < nums.Length; i++)
    {
        long need = (long)target - nums[i];
        if (need >= int.MinValue && need <= int.MaxValue &&
            seen.TryGetValue((int)need, out int j))
            return new[] { j, i };
        seen[nums[i]] = i;
    }
    return Array.Empty<int>();
}
```

Explain: Assume non-null input; return indices for any valid pair, or an empty array if absent. [2,7,11,15], target 9 gives [0,1]. [3,3], target 6 gives [0,1]. Check before inserting so the same index is not reused. Average $O(n)$ time and $O(n)$ space. The long subtraction avoids int overflow.

Follow-up: Why a Dictionary rather than HashSet? We need the earlier index, not only whether the complement exists.

Practice: rewrite it without looking; predict one normal case and one edge case. Snippets assume normal namespace imports and referenced packages/types.

Worked example 02 / LINQ: paid orders by customer

```
var totals = orders
    .Where(o => o.Status == "Paid")
    .GroupBy(o => o.CustomerId)
    .Select(g => new {
        CustomerId = g.Key,
        Total = g.Sum(o => o.Amount)
    })
    .OrderByDescending(x => x.Total)
    .ToList();
```

Explain: Assume orders is an in-memory collection with CustomerId, Status and decimal Amount. For paid amounts 10 and 20 for customer A, total is 30; unpaid orders are excluded. GroupBy creates groups; Select changes the shape. An EF query requires checking provider translation.

Follow-up: What changes if orders is IQueryable? The expression may be translated to database operations before materialization.

Practice: rewrite it without looking; predict one normal case and one edge case. Snippets assume normal namespace imports and referenced packages/types.

Worked example 03 / Async HTTP with cancellation

```
public async Task<string> LoadAsync(
    HttpClient client, CancellationToken ct)
{
    using var response = await client.GetAsync(
        "https://example.com/api/catalog", ct);
    response.EnsureSuccessStatusCode();
    return await response.Content.ReadAsStringAsync(ct);
}
```

Explain: The endpoint URL is illustrative. The caller supplies a configured client and owns its lifetime; this method owns the response. Non-success statuses throw. Cancellation propagates. In a real API, map dependency failures deliberately and configure timeouts.

Follow-up: Why not Task.Run? The HTTP library already provides asynchronous I/O. Why not dispose client here? This method does not own it.

Practice: rewrite it without looking; predict one normal case and one edge case. Snippets assume normal namespace imports and referenced packages/types.

Worked example 04 / EF Core: read-only page

```
var page = await db.Products
    .AsNoTracking()
    .Where(p => p.IsActive)
    .OrderBy(p => p.Id)
    .Skip((pageNumber - 1) * pageSize)
    .Take(pageSize)
    .Select(p => new { p.Id, p.Name, p.Price })
    .ToListAsync(ct);
```

Explain: Assume an EF DbContext with Products and validated bounded integers pageNumber ≥ 1 and $1 \leq \text{pageSize} \leq 100$. Ordering is stable because Id is unique. Filtering and projection happen before materialization. For deep pages consider keyset pagination, such as $\text{Id} > \text{lastId}$.

Follow-up: Do you need AsNoTracking for this scalar-only projection? No entities are materialized here, so there are no entity instances to track; it mainly communicates read-only intent.

Practice: rewrite it without looking; predict one normal case and one edge case. Snippets assume normal namespace imports and referenced packages/types.

Worked example 05 / SQL: customers with three paid orders

```
SELECT CustomerId, COUNT(*) AS PaidOrders,  
       SUM(Amount) AS TotalPaid  
FROM Orders  
WHERE Status = 'Paid'  
GROUP BY CustomerId  
HAVING COUNT(*) >= 3  
ORDER BY TotalPaid DESC, CustomerId;
```

Explain: Assume one row per order and non-null Amount. WHERE first keeps paid orders. GROUP BY builds one group per customer. HAVING retains groups with at least three orders. The second ordering key makes ties deterministic.

Follow-up: If you need customers with zero orders, why is this query insufficient? Start from Customers and join Orders with suitable counting and filtering.

Practice: rewrite it without looking; predict one normal case and one edge case. Snippets assume normal namespace imports and referenced packages/types.

Worked example 06 / Polymorphism and injection

```
public interface IDiscount
{
    decimal Apply(decimal amount);
}
public sealed class TenPercent : IDiscount
{
    public decimal Apply(decimal amount) => amount * 0.90m;
}
public sealed class Checkout
{
    private readonly IDiscount discount;
    public Checkout(IDiscount discount) => this.discount = discount;
    public decimal Total(decimal amount) => discount.Apply(amount);
}
// Registration in Program.cs:
// builder.Services.AddScoped<IDiscount, TenPercent>();
// builder.Services.AddScoped<Checkout>();
```

Explain: Assume non-negative amounts; real pricing also defines currency rounding and validation. Checkout depends on the contract rather than a concrete policy. TenPercent.Apply(100m) returns 90m. A different implementation changes the policy without changing Checkout.

Follow-up: Which SOLID principles does this illustrate? Dependency inversion and open/closed design. Explain why you would avoid unnecessary abstraction for trivial, unchanging logic.

Practice: rewrite it without looking; predict one normal case and one edge case. Snippets assume normal namespace imports and referenced packages/types.

A 30-minute mock round

2 minutes: introduce yourself using only real experience.

4 minutes: explain DI lifetimes and fix a singleton that captures DbContext.

5 minutes: trace an authenticated request and explain 401 versus 403.

6 minutes: implement Two Sum and discuss duplicates, no match and complexity.

5 minutes: diagnose an endpoint that loads 100 orders and runs 100 additional queries.

4 minutes: explain async/await, cancellation and why one DbContext cannot run parallel queries.

4 minutes: describe a real production issue and ask two questions about the role.

Self-check: For each response give yourself 0 = cannot explain, 1 = definition only, 2 = example, 3 = trade-off and edge case. Revisit the lowest scores; do not spend the final hour rereading what you already know.

Last-minute recall sheet

- Scoped is per scope, usually per HTTP request; singleton is shared and must be thread-safe.
- await does not mean a new thread; avoid blocking async calls.
- DbContext is short-lived and not thread-safe.
- Authentication identifies; authorization permits; always check resource ownership.
- JWT signing is not encryption; validate issuer, audience, signature and lifetime.
- DTOs prevent accidental contract exposure; still enforce authorization server-side.
- Filter and project before ToList; measure SQL, indexes and N+1 calls.
- WHERE filters rows; HAVING filters groups; LEFT JOIN preserves unmatched left rows.
- Transactions preserve a unit of work; retry decisions must respect idempotency.
- Use throw; when rethrowing; never expose internal errors to clients.
- GC handles reachable managed objects; using handles deterministic disposal.
- Describe your actual contribution, evidence and lessons learned.

When you freeze: "Let me clarify the requirement. I would start with a simple correct solution, then discuss performance and failure cases."

Official references

Checked 7 October 2026. The Q&A; and examples are original teaching material. Links below support the specific framework details flagged in the answers.

A | C# asynchronous programming

[Read Microsoft Learn documentation](#)

B | ASP.NET Core middleware

[Read Microsoft Learn documentation](#)

C | ASP.NET Core dependency injection

[Read Microsoft Learn documentation](#)

D | EF Core efficient querying

[Read Microsoft Learn documentation](#)

E | EF Core asynchronous operations

[Read Microsoft Learn documentation](#)

F | EF Core concurrency conflicts

[Read Microsoft Learn documentation](#)

Version note: ask which .NET and EF Core versions the company uses. Some API conventions and capabilities vary by version and database provider. This pack emphasizes stable concepts rather than version-specific trivia.